

Agenda

① Intro to Structural
Design Patterns

② Adapter Design Pattern

③ Facade Design Pattern

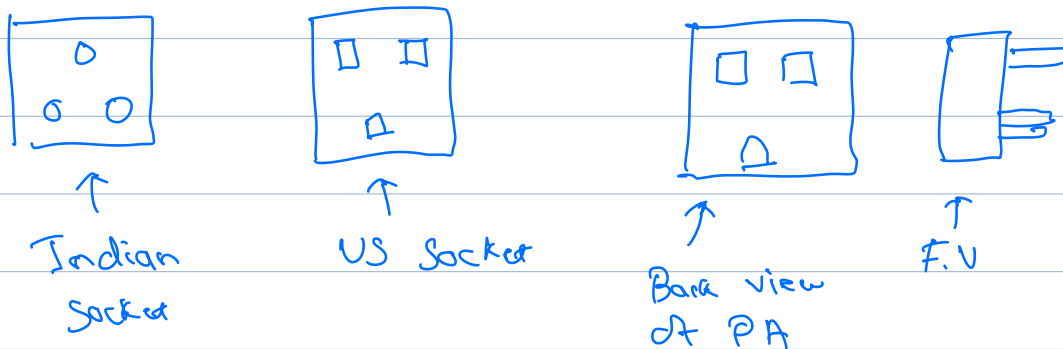
1] Intro to Structural Design Patterns

Deal with structures of your classes.

Helps us structure our classes & objects in a way so that we can create more efficient & larger structures.

2] Adapter Design Pattern

↳ helps us connect 2 things



He will use adapter design pattern to make 2 incompatible code bases work with each other.

1st is our code base & 2nd thing would be a library.

He are building Zomato & we want to send emails to our customers, we will use Sendgrid for this initially.

~~SOLID~~

class Zomato {

~~Sendgrid~~ client;
mailgun

public void sendEmail (String content,
String to, String from) {

client.sendEmail (content, to, from,
null, null);

.send (to, from, content,
null, null);

3

3

Steps to fix DIP / Steps to implement adapter :

Step #1: Introduce an interface / abstract class called as Email Adapter.

```
interface EmailAdapter {
```

```
    void sendEmail (String to, String  
                    from, String content);
```

```
}
```

Step #2: Whichever library you want to use create a class for it & make it implement the Email Adapter interface.

```
class SendgridAdapter implements  
    EmailAdapter {
```

```
    Sendgrid client;
```

```
    void sendEmail (String to, String  
        from, String content) {
```

```
        client.sendEmail (content, to,  
            from);
```

```
}
```

```
class MailgunAdapter  
    implements EA {
```

```
    Mailgun client
```

```
    sendEmail (        ) {
```

```
}
```

```
    client.send (to, from,
```

```
        content);
```

```
}
```

Step #3: In class Zomato, create

an attr of type Email Adapter
& put an object any of
its children via dependency
injection or via hardcoding.

```
class Zomato {
```

```
    EmailAdapter adapter = new Sendgrid  
                                Adapter();
```

```
    public void sendEmail (String to,  
                           String from, String content) {
```

```
        adapter.sendEmail ( to, from,  
                             content  
                             );
```

```
}
```

```
}
```

Real life Example

```
class PhonePe {
```

```
YesBankApi api;
```

```
BankApiAdapter
```

```
public void sendMoney (String from,  
String to, double amt) {
```

```
api.sendMoney (from, to,  
amt);
```

3

3

Adapter
interface BankApi {

void sendMoney ();

void checkBal ();

3

class YesBankAdapter

implements BankApi Adapter {

YesBankApi api

class HDFCBank

Adapter implements

BankApi Adapter {

HDFCBankApi

api ;

3

3

Facade Design Pattern

↳ structure that hides inner details of that structure

```
class Amazon {
```

```
    public void placeOrder ( ) {  
        inventoryService.updateInventory();  
        orderService.createOrder ( );  
        sellerService.sendOrderDetails();  
        transportService.sendOrderDetails();  
        customerService.sendInvoice();  
    }
```

```
}
```

```
}
```

Facade Design Pattern:

Class Amazon Helper :

```
public void placeOrder ( ) {  
    inventoryService. update Inventory(),  
    orderService. create Order ( ) ;  
    sellerService. send Order Details();  
    transport Service. send Order Details()  
    CustomerService. send Invoice();  
}
```

3

3

```
class Amazon {
```

```
    public void placeOrder () {  
        AmazonHelper helper = new  
        AmazonHelper ();
```

```
        helper.placeOrder ();
```

```
    }
```

```
}
```

Goal for next class: Tuesday

↳ Singleton +	(3)
Builder	(3)
	6

Refactoring.guru